

NUR Hand-in 1

Sam Beckers, s2963108

February 21, 2025

Abstract

This document contains the code and outputs of Hand-in 1 of the course Numerical recipes for astrophysics.

1 Exercise 1

The Poisson distribution is given by

$$P_{\lambda}(k) = \frac{\lambda^k e^{-\lambda}}{k!} \quad (1)$$

with k integer and the mean λ is a positive number. The factorial of k will produce very large floating point numbers which causes an **Overflow** error. Therefore, we can also calculate the log of $P_{\lambda}(k)$, which can be written as (according to WolframAlpha)

$$k \log(\lambda) - \lambda - \log(\Gamma(1 + k)) \quad (2)$$

where $\Gamma(1 + k) = (1 + k - 1)! = k!$. We can approximate the Gamma function by the Stirling approximation,

$$\Gamma(1 + k) = k! \approx \sqrt{2\pi k} \left(\frac{k}{e}\right)^k. \quad (3)$$

However, this still results in an overflow for larger values of k . Therefore, we switched approach and did not use the Gamma function. Instead, we used Ramanujan's approximation for $\log(k!)$:

$$\log(k!) \approx k \log(k) - k + \frac{\log(k(1 + 4k(1 + 2k)))}{6} + \frac{\log(\pi)}{2} \quad (4)$$

An alternate form of the log of $P_{\lambda}(k)$ is

$$k \log(\lambda) - \lambda - \log(k!) \quad (5)$$

Combining this with Equation 4 gives:

$$\log(P_{\lambda}(k)) = k \log(\lambda) - \lambda - k \log(k) + k - \frac{\log(k(1 + 4k(1 + 2k)))}{6} - \frac{\log(\pi)}{2} \quad (6)$$

In the code, we first try to use Equation 1 as this does not overflow for small k values. If it overflows, we use Equation 6 and simply return $\exp(\log(P_{\lambda}(k)))$. NumPy's `exp()` function can also overflow, and instead of raising an error it will return zero or infinity. In this case, the value of P just contains too many digits, and the only way to get a reasonable output is to use the Decimal module, which can handle these situations. However, this only happens in extreme cases, and we put this as an extra in code.

We make sure that all k and λ values we pass to the Poisson distribution and factorial function are `float32s`. In the factorial function, we round k to make it an integer.

See the code below for our implementation of this exercise:

```

1  """
2  NUR_Handin1.1
3  Created on 14-02-2025
4
5  @author(s): Sam Beckers
6  """
7  import numpy as np
8  import itertools
9  import decimal
10 import warnings
11 warnings.filterwarnings("ignore")
12
13 def factorial(x):
14     """Simple factorial function
15
16     Args:
17         x (float): input value
18
19     Returns:
20         f: factorial of x
21     """
22     f = 1
23     for i in range(1, round(x)+1):
24         f *= i
25     return f
26
27 def poisson(lam, k):
28     """Poisson distribution function
29
30     Args:
31         lam (float32): lambda value
32         k (float32): k value
33
34     Returns:
35         P: P(lamda, k)
36     """
37     try:
38         P = lam**k * np.exp(-lam) / factorial(k) # Poisson equation
39         if P == np.inf or P == 0.0: # Check for overflow
40             raise OverflowError
41         else:
42             return P
43     except OverflowError: # In the case of overflow, use Ramanujan's approximation for
44         log_P = k * np.log(lam) - lam - k * np.log(k) + k - np.log(k*(1+(4*k*(1+2*k))))
45         / 6 - np.log(np.pi) / 2
46         if np.exp(log_P) == 0.0 or np.exp(log_P) == np.inf:
47             return np.exp(decimal.Decimal(log_P)) # if P overflows, we cannot prevent
48             this except with the Decimal module
49         elif log_P == np.inf: # If log_P overflows, something is wrong with the
50             approximation
51             raise OverflowError
52         else:
53             return np.exp(log_P)
54
55 # Test values (include k=339 to show overflow handling of P)
56 lam_arr = np.array([1,5,3,2.6,100,101, 1], dtype=np.float32)
57 k_arr = np.array([0,10,21,40,5,200, 339], dtype=np.float32)
58
59 print("Results for the given lambda and k values:")
60 for lam, k in zip(lam_arr, k_arr):
61     print(f"lambda = {lam}, k = {k}, P = {poisson(lam, k)}")
62
63 # More test values
64 lam_extended = np.linspace(1, 101, 1000, dtype=np.float32)
65 k_extended = np.linspace(0, 200, 1000, dtype=np.float32)
66
67 P_extended = []
68 for lam, k in itertools.product(lam_extended, k_extended): # Create all combinations of

```

```

66     lambda and k
67     P_extended.append(poisson(lam, k))
68 count = 0
69 for p in P_extended:
70     if p == 0.0 or p < 0.0:
71         count += 1
72
73 print(f"\nPercentage of zero or negative values in {len(P_extended)} combinations of
lambda and k: {count/len(P_extended)*100}%")

```

NUR_Handin_1.1.py

As an additional test of the robustness of our algorithm, we also tested 10000 combinations of values between $\lambda = 1 - 101$ and $k = 0 - 200$.

The result of the code is shown here:

```

1 Results for the given lambda and k values:
2 lambda = 1.0, k = 0.0, P = 0.3678794503211975
3 lambda = 5.0, k = 10.0, P = 0.018132789248511905
4 lambda = 3.0, k = 21.0, P = 1.0193398552911732e-11
5 lambda = 2.5999999046325684, k = 40.0, P = 3.615119275388806e-33
6 lambda = 100.0, k = 5.0, P = 3.152921508857598e-36
7 lambda = 101.0, k = 200.0, P = 1.2695652632439885e-18
8 lambda = 1.0, k = 339.0, P = 2.452109834638716400437827337E-713
9
10 Percentage of zero or negative values in 1000000 combinations of lambda and k: 0.0%

```

NUR_Handin_1.1_output.txt